

CPP Module 08

Templated containers, iterators, algorithms

C++98 • École 42 • Présentation de soutenance

Exercice	Sujet	Notion clé
ex00	easyfind	Fonction template + std::find
ex01	Span	Classe + méthode template + algorithmes STL
ex02	MutantStack	Héritage de conteneur STL + itérateurs

Notions Fondamentales

1. Les Templates (Generiques)

Un **template** permet d'ecrire du code **generique** qui fonctionne avec n'importe quel type. Le compilateur genere automatiquement la version concrete de la fonction ou de la classe au moment de la compilation, selon le type utilise.

■ Syntaxe d'une fonction template

```
template <typename T>
T maFonction(T parametre) { ... }

// Le compilateur genere automatiquement :
int maFonction(int parametre) // si T = int
double maFonction(double parametre) // si T = double
```

Regles importantes :

- L'implementation d'un template doit etre dans le **.hpp** (pas dans le **.cpp**), car le compilateur a besoin de voir le code complet au moment de l'instantiation.
- On peut avoir une methode template dans une classe non-template (comme dans Span avec addRange).

2. Les Iterateurs

Un **iterateur** est un objet qui **pointe vers un element** d'un conteneur. Il abstrait le mode d'accès aux données : peu importe si le conteneur est un tableau, une liste chaînée ou une pile, l'interface est identique.

Expression	Signification
<code>container.begin()</code>	Iterateur sur le PREMIER element
<code>container.end()</code>	Iterateur apres le DERNIER element (sentinelle)
<code>*it</code>	Dereference : obtient la valeur pointee
<code>++it</code>	Avance vers l'element suivant
<code>it != container.end()</code>	Condition de fin de parcours

3. Les Conteneurs STL

Conteneur	Description	Acces
<code>std::vector<T></code>	Tableau dynamique, elements contigus en memoire	O(1) aleatoire
<code>std::list<T></code>	Liste doublement chaine	O(n) lineaire
<code>std::deque<T></code>	Double-ended queue, ajout rapide aux deux bouts	O(1) aleatoire

<code>std::stack<T></code>	Adaptateur LIFO (Last In First Out), base sur deque	Pas d'iterateurs
----------------------------------	---	------------------

EX00 — easyfind

Implementer une **fonction template** `easyfind` qui cherche un entier dans n'importe quel conteneur STL séquentiel et retourne un itérateur sur l'élément trouvé, ou lève une exception s'il est absent.

Signature

```
template <typename T>
typename T::iterator easyfind(T &container, int value);
```

Implementation

```
// std::find parcourt [begin, end) et retourne un itérateur
// sur le premier élément égal à value, ou end() si absent
typename T::iterator it = std::find(container.begin(), container.end(), value);

if (it == container.end())
    throw std::runtime_error("Value not found in container");

return it;
```

■ Pourquoi 'typename T::iterator' ?

T est un type inconnu au moment de l'écriture du template.
T::iterator pourrait être soit un TYPE soit un membre de données.
Le mot-cle `typename` indique au compilateur que c'est un TYPE.
Sans ce mot-cle, le compilateur refuserait de compiler.

std::find — Algorithme STL cle

Parametre	Role
<code>begin</code>	Itérateur début de la plage
<code>end</code>	Itérateur fin (exclu)
<code>value</code>	Valeur à rechercher
<code>retour</code>	Itérateur sur l'élément trouvé, ou <code>end()</code> si absent

Exemple d'utilisation

```
std::vector<int> v;
v.push_back(1); v.push_back(2); v.push_back(3);

try {
    std::vector<int>::iterator it = easyfind(v, 2);
```

```
std::cout << *it << std::endl; // affiche : 2
} catch (std::exception &e) {
std::cout << e.what() << std::endl;
}
```

EX01 — Span

Créer une **classe Span** qui stocke jusqu'à N entiers et peut calculer le plus court (*shortestSpan*) et le plus long (*longestSpan*) écart entre ses éléments. La classe doit aussi accepter l'ajout en masse via une méthode template `addRange`.

Forme Canonique Orthodoxe

Toute classe 42 doit implémenter les **4 membres obligatoires** :

```
class Span
{
public:
    Span(unsigned int N); // 1. Constructeur parametre
    Span(const Span &other); // 2. Constructeur de copie
    Span &operator=(const Span &other); // 3. Operateur d'affectation
    ~Span(); // 4. Destructeur

    // Methode template dans une classe normale :
    template <typename Iterator>
    void addRange(Iterator begin, Iterator end) { ... }

private:
    unsigned int _maxSize; // capacite max
    std::vector<int> _numbers; // les entiers stockes
};
```

Algorithme `shortestSpan`

■ Pourquoi trier avant de chercher le minimum ?

La plus petite différence entre deux éléments QUELCONQUES se trouve toujours entre deux éléments ADJACENTS dans le tableau trié.

Exemple : [17, 3, 9, 6, 11] → trié : [3, 6, 9, 11, 17]

Différences adjacentes : 3, 3, 2, 6 → minimum = 2

```
int Span::shortestSpan() const
{
    std::vector<int> sorted(_numbers); // copie pour ne pas modifier l'original
    std::sort(sorted.begin(), sorted.end()); // tri croissant

    int minSpan = sorted[1] - sorted[0];
    for (size_t i = 1; i < sorted.size() - 1; ++i)
        minSpan = std::min(minSpan, sorted[i+1] - sorted[i]);
    return minSpan;
}
```

Algorithme longestSpan

```
int Span::longestSpan() const
{
    int maxVal = *std::max_element(_numbers.begin(), _numbers.end());
    int minVal = *std::min_element(_numbers.begin(), _numbers.end());
    return maxVal - minVal;
}
```

addRange — Methode template

Permet d'ajouter des elements depuis **n'importe quelle source** qui fourni des iterateurs (vector, list, tableau C...) :

```
template <typename Iterator>
void addRange(Iterator begin, Iterator end)
{
    for (Iterator it = begin; it != end; ++it)
        addNumber(*it); // *it = valeur pointee
}

// Utilisation :
Span bigSpan(10000);
std::vector<int> nums; // rempli avec 0..9999
bigSpan.addRange(nums.begin(), nums.end());
```

EX02 — MutantStack

Créer une classe template **MutantStack** qui se comporte exactement comme `std::stack` (push, pop, top, size, empty) MAIS qui ajoute des **itérateurs**. La solution : hériter de `std::stack` et exposer les itérateurs du conteneur interne.

Pourquoi `std::stack` n'a pas d'itérateurs ?

■ `std::stack` est un adaptateur de conteneur

`std::stack` est une INTERFACE au-dessus d'un autre conteneur (par défaut `std::deque`). Il cache intentionnellement les itérateurs pour forcer un accès LIFO strict (Last In First Out). Seuls `push()`, `pop()`, `top()`, `size()`, `empty()` sont exposés.

Structure de MutantStack

```
template <typename T>
class MutantStack : public std::stack<T>
{
public:
    // Alias pour le type d'itérateur du conteneur interne
    typedef typename std::stack<T>::container_type::iterator iterator;
    typedef typename std::stack<T>::container_type::const_iterator const_iterator;

    // "c" est le membre PROTEGE de std::stack : le conteneur interne
    iterator begin() { return std::stack<T>::c.begin(); }
    iterator end() { return std::stack<T>::c.end(); }
};
```

Le membre protégé 'c'

■ Accès à 'c' via l'héritage

`std::stack<T>` déclare son conteneur interne comme :
`protected: Container c; // accessible depuis les classes dérivées`

Parce que `MutantStack` hérite de `std::stack`, on peut accéder à "c" directement.
`c.begin()` et `c.end()` donnent les itérateurs de la deque sous-jacente.

Exemple d'utilisation

```
MutantStack<int> mstack;
mstack.push(5); // pile : [5]
mstack.push(17); // pile : [5, 17]
```



```
mstack.push(3); // pile : [5, 17, 3]

// Parcours avec itérateurs (du fond vers le sommet)
MutantStack<int>::iterator it = mstack.begin();
MutantStack<int>::iterator ite = mstack.end();
while (it != ite)
    std::cout << *it++ << std::endl; // affiche : 5, 17, 3
```

Recap — Fonctions STL Utilisees

Fonction	Header	Role	Retour
<code>std::find</code>		Cherche une valeur dans [begin, end)	Iterateur sur l'element ou end()
<code>std::sort</code>		Trie un conteneur en ordre croissant	<code>void</code>
<code>std::min_element</code>		Trouve le plus petit element	Iterateur sur le min
<code>std::max_element</code>		Trouve le plus grand element	Iterateur sur le max

Questions Frequentes en Soutenance

Q : Pourquoi l'implementation du template est dans le .hpp ?

→ Le compilateur genere le code concret au moment de l'instantiation. Il a besoin de voir l'implementation complete pour faire ca. Si elle est dans un .cpp separe, le compilateur ne peut pas la trouver.

Q : Pourquoi utiliser 'typename T::iterator' et pas juste 'T::iterator' ?

→ Parce que T est un type inconnu (dependant), le compilateur ne sait pas si T::iterator est un type ou un membre de donnees. Le mot-cle 'typename' lui dit explicitement que c'est un type.

Q : Pourquoi trier avant shortestSpan ?

→ La plus petite difference entre deux elements quelconques se trouve toujours entre deux elements adjacents apres tri. Sans tri, il faudrait comparer toutes les paires : $O(n^2)$ au lieu de $O(n \log n)$ avec le tri.

Q : Pourquoi heriter de std::stack et pas l'encapsuler ?

→ L'heritage permet de reutiliser toutes les methodes (push, pop, top, size, empty) sans les reecrire. Et via l'heritage, on a acces au membre protege 'c' qui contient le conteneur interne, ce qui permet d'en obtenir les iterateurs.

Q : Que se passe-t-il si on appelle shortestSpan avec 0 ou 1 element ?

→ La methode leve une exception `std::runtime_error`. On verifie `_numbers.size() <= 1` au debut de la methode et on throw si c'est le cas.

Q : addRange peut-elle lancer une exception ?

→ Oui. Elle appelle `addNumber()` pour chaque element. Si le Span est plein avant la fin de la plage, `addNumber()` levera `std::runtime_error`.

Schemas Visuels

Fonctionnement d'un itérateur

```
Conteneur : [ 10 | 20 | 30 | 40 | 50 ]
↑ ↑
begin() end() (sentinelle, hors tableau)

it = begin() → *it = 10
++it → *it = 20
++it → *it = 30
++it → *it = 40
++it → *it = 50
++it → it == end() (sortie de boucle)
```

MutantStack — Architecture interne

```
MutantStack<int>
■■■ herite de std::stack<int>
■ ■■■ push(), pop(), top(), size(), empty()
■ ■■■ protected: std::deque<int> c ← conteneur interne
■ [ 5 | 3 | 5 | 737 | 0 ]
■ ↑ ↑
■ fond sommet (top)
■■■ ajoute :
begin() → c.begin() (fond de la pile)
end() → c.end() (apres le sommet)
```

shortestSpan — Algorithme visuel

```
Entree : [17, 3, 9, 6, 11]

Etape 1 – Tri : [3, 6, 9, 11, 17]
↑ ↑ ↑ ↑ ↑
Etape 2 – Differences: 3 3 2 6
↑
Etape 3 – Minimum : 2 ← shortestSpan

longestSpan : max - min = 17 - 3 = 14
```

Commandes de Build

```
# Compiler et tester chaque exercice  
cd ex00 && make && ./easyfind  
cd ex01 && make && ./span  
cd ex02 && make && ./mutantstack  
  
# Flags utilises : -Wall -Wextra -Werror -std=c++98
```